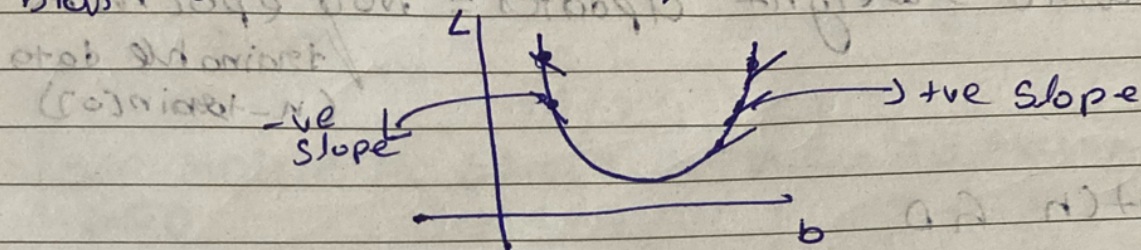


time Gradient Descent in Neural Network

Way to find most optimized solution

Is a way to minimize loss function of all weights & bias



Learning rate $\rightarrow$ Step to reach minimum

Back-propagation Algorithm

Epoch = 5

for i in range (epoch)

for j in range(n.shape[0]):

: (w, b) $\rightarrow$ Split 1 row (random)

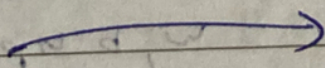
$\rightarrow$ Predict (using forward pass)

$\rightarrow$ calculate loss

$\rightarrow$ update weights & bias w

hd

Gradient Descent



$$w_n = w_0 - \eta \frac{dL}{dw}$$

$\rightarrow$ calculate avg loss

Batch / Stochastic / Mini Batch

Batch $\rightarrow$ weight updates = No. of epochs

Stochastic $\rightarrow$ weight updates = No. of epochs $\times$
 / trainable data
 (n - train[0])

Batch GD

for i in range(epochs)

$\hat{y} \rightarrow \text{np.dot}(x, w) + b$

loss

$\downarrow$
Avg

Stochastic

for i in range(epochs):

for j in range(n - train_start):

i = random point

$\hat{y}$

loss

w, b update

$\downarrow$

Avg loss print $\rightarrow$ for the epoch

frequency of weights update higher

which is faster (given some no. of epochs).

Batch gradient descent

which is the faster to converge (given some epoch)

SGD

loss reduce is stable in Batch & rather than SGD

→ SGD is preferred for finding global minima because of unsteady movement

→ we get approx solution SGD

Vectorization

↳ we use dot product in BGD to calculate $\hat{y} = \text{np.dot}(X_{\text{train}}, w)$

rather than having loop finding for each value, dot product is optimized way, so it's faster. this known as vectorization

but it fails on very big datasets because of data on memory is inefficient

Mini Batch GD

↳ lies between Batch GD & SGD

Here we decide batch size & divider data accordingly

for i in range (epochs):

for j in num of batches

batch →

hp. dot

used (vectorization)

↓

update

fast

BGD > MBGD > SGD

fast convergence

SGD > MBGD > BGD

Why Batch size is provided in multiple of 2?

2, 4, 8, 32, 64, ...

↓

RAM effectiveness

→ Binary

optimization technique

make faster



What if batch size doesn't divide #groups properly

now $S \rightarrow n = 400$

Batch size = 150

$$\text{Batch} = \frac{400}{150} = 2.66$$

1st Batch



150

2nd Batch



150

3rd Batch



100